

For the model's capabilities, API changes, pricing, and availability, see [What's new in Claude Table 5.1](#). For techniques that apply across Claude models, see [Prompting best practices](#).

Your existing Claude Table 5 prompts should perform well on Claude Table 5.1 without changes, but a handful of behavioral differences are worth knowing about. Start with the section that matches what you observe:

- Unsure which effort level to run, or latency and cost are higher than the task warrants: [Consider all effort levels](#)
- Little or no text between tool calls: [Ask for user-facing progress updates](#)
- One tool call per turn in agent loops: [Batch independent tool calls in agent loops](#)
- Requests fail with `bound to a different conversation`, or your harness edits earlier turns between requests: [Keep the conversation history append-only](#)
- Prose runs long and dense: [Writing density](#)
- Chat replies carry less structure than the content needs: [Formatting in chat](#)
- Summaries reproduce source wording without marking it as a quotation: [Quoting retrieved sources](#)
- Turn ends before the work is done, or the model asks permission for work you already requested: [Finish the whole task](#)
- Client-side compaction summaries drop constraints, decisions, or exact details: [Tell the model what to preserve in compaction summaries](#)
- Unrequested fixes or extensions, or more committed test files than the task called for: [Keep changes and tests to what the task asks for](#)
- Answers from memory instead of searching at low effort: [Search triggering at low effort](#)
- Benign coding requests return `stop_reason: "refusal"`: [Reduce safeguard false positives](#)
- Whole files rewritten for small changes: [Prefer targeted edits over whole-file rewrites](#)
- Long deliverables at `xhigh` or `max` effort take a long time or hit `max_tokens`: [Leave room for long outputs at xhigh and max effort](#)
- Lead agent idles while subagents run: [Let the lead agent keep working while subagents run](#)
- Answers about charts and dense images miss detail: [Give vision work tools to crop and zoom](#)

Consider all effort levels

Start at the default [effort](#) level, `high`, then test the other levels (`low`, `medium`, `xhigh`, and `max`) against your own evals. Effort is the primary control for trading off intelligence, latency, and cost on Claude Table 5.1. Re-run the sweep even if you already ran one on Claude Table 5: effort level names don't correspond to the same amount of thinking across models.

Claude Fable 5.1's capability gains over Claude Fable 5 show up across effort levels and are largest at the higher settings. At `medium`, results roughly match Claude Fable 5 at lower cost, so step down to `medium` or `low` where your evals show quality holds. At `low`, Claude Fable 5.1 is often competitive with Claude Opus and Claude Sonnet models on cost per task while scoring higher, so include it in the comparison wherever you'd otherwise run a smaller model at a higher effort level.

Two effort-specific behaviors have their own sections: at `low`, Claude Fable 5.1 calls search and retrieval tools less often (see [Search triggering at low effort](#)), and at `xhigh` and `max` it can think for longer before writing a long deliverable (see [Leave room for long outputs at xhigh and max effort](#)).

Ask for user-facing progress updates

Claude Fable 5.1's default behavior is to write fewer user-facing updates during long tool-calling turns than Claude Fable 5 does. This becomes more pronounced at higher effort and in longer tool chains. Users see the agent go quiet for minutes at a time, or a final message that covers only the last step rather than the whole task.

First, check that your client receives progress updates at all. The model's short notes between tool calls, what it just found and what it's doing next, come back as [progress-update](#) `thinking` blocks, and those blocks are empty under the default `thinking.display` of `"omitted"`. Set `display: "updates"` (beta, `thinking-display-updates-2026-08-18` header) and render each non-empty `thinking` block as a status line, or set `"summarized"` to receive them along with summarized reasoning. If you aren't requesting them, the model's updates may simply not be reaching your users.

Second, audit your prompt for instructions that suppress narration. Some earlier models were eager to give updates while working, which led to system prompt lines such as "hold all findings for the final response." Remove lines like that before adding anything.

If you still want more updates, for example when pair programming or in other human-in-the-loop work, add a short system prompt line that says when you want user-facing text from the model and what each update should contain:

```
Before you start, say in a line what you're about to do; brief updates while
you work help the user follow along. Close with a short recap that stands on
its own – what you found, what you did, and what's next – so a reader who only
sees the last message has the full picture.
```

If your product collapses or hides tool output, tell the model. Otherwise it may run commands to "show" the user output that your UI never displays. Deliver the note in a [turn-scoped system message](#) (`clear_at: "next_user_message"`, beta):

```
Only you see that command's output – the user's terminal shows at most a few
```

lines of it. If the user needs to read any of it, put it in your reply.

Batch independent tool calls in agent loops

Claude Fable 5.1 usually issues parallel tool calls as expected: when a request names several things to fetch, it issues those calls in parallel. The exception is coding and computer-use loops where the next independent calls are implied by the task rather than explicitly requested (custom coding agents, bash-and-editor harnesses, computer use): there it may issue them one per turn instead. This doesn't affect answer quality, but each extra turn costs tokens, a round trip, and wall-clock time. A one-sentence nudge at the end of the current request addresses it:

First privately list what you need next; then request every item that doesn't depend on another's result in this one response.

Each time you send tool results back, append it after that user message as a [turn-scoped system message](#): a `role: "system"` entry in `messages` with `clear_at: "next_user_message"`. Once a later user message exists, the API clears the earlier copies, so the model reads only the newest one. Turn-scoped system messages are in beta and require the [beta header](#) `mid-conversation-system-clear-at-2026-08-21`. Without the beta, place the sentence in a text block after the `tool_result` blocks in the same user message instead.

Append a fresh copy each turn and leave the earlier copies where they are, byte-for-byte. They stay in the array, but once cleared the model doesn't see them and they cost no input tokens. Deleting or rewriting them is an edit to earlier turns: it restarts the [prompt cache](#) from that point and invalidates the thinking blocks that came after them (see [Keep the conversation history append-only](#)).

The following loop shows this placement. Each assistant turn goes back exactly as returned, each user turn carries only the tool results, and a fresh turn-scoped copy of the nudge follows it.

Python

```
import anthropic
from anthropic.types.beta import (
    BetaMessageParam,
    BetaToolParam,
    BetaToolResultBlockParam,
)

client = anthropic.Anthropic()

BATCH_NUDGE = (
    "First privately list what you need next; then request every item "
    "that doesn't depend on another's result in this one response."
```

```

)
# In-memory files stand in for a working directory so the sample runs anywhere.
FILES = {
    "pyproject.toml": """\
[project]
name = "demo"
version = "0.1.0"
description = "Demo project for the batching example"
""",
    "README.md": """\
# demo

A small demo project. Run `demo --help` for usage.
""",
}
tools: list[BetaToolParam] = [
    {
        "name": "read_file",
        "description": "Read a UTF-8 text file from the working directory.",
        "input_schema": {
            "type": "object",
            "properties": {"path": {"type": "string"}},
            "required": ["path"],
        },
    },
]
messages: list[BetaMessageParam] = [
    {"role": "user", "content": "Summarize pyproject.toml and README.md."}
]

while True:
    response = client.beta.messages.create(
        model="claude-fable-5-1",
        max_tokens=16000,
        betas=["mid-conversation-system-clear-at-2026-08-21"],
        tools=tools,
        messages=messages,
    )
    # Append the assistant turn exactly as returned, thinking blocks included.
    messages.append({"role": "assistant", "content": response.content})
    if response.stop_reason != "tool_use":
        break
    tool_results: list[BetaToolResultBlockParam] = []
    for block in response.content:
        if block.type == "tool_use":
            raw_path = block.input.get("path")
            path = raw_path if isinstance(raw_path, str) else ""
            if path in FILES:
                tool_results.append(
                    {
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": FILES[path],
                    }
                )

```

```

        }
    )
    else:
        tool_results.append(
            {
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": f"File not found: {path}",
                "is_error": True,
            }
        )
    # Send the tool results as the user turn, then a fresh copy of the nudge as
a
    # turn-scoped system message. Leave earlier copies in place: the API clears
    # so the model sees only the newest one.
    messages.append({"role": "user", "content": tool_results})
    messages.append(
        {"role": "system", "content": BATCH_NUDGE, "clear_at":
"next_user_message"}
    )

print(next((block.text for block in response.content if block.type == "text"),
""))

```

Keep the conversation history append-only

Append each assistant turn to the history exactly as the API returned it, thinking blocks included, and don't edit earlier turns between requests. For new accounts created on or after August 31, 2026, Claude Fable 5.1's thinking blocks are valid [only in the exact conversation that produced them](#): a request that replays a thinking block after its prefix (the system prompt, the tool list, or any earlier message) has changed returns a 400, or drops the affected blocks if you set `thinking.block_binding.prefix_mismatch_behavior: "drop_block"` (beta, `thinking-binding-controls-2026-08-01` header). Adopt the pattern even if the check isn't enforced for your account, so the same code works on every account.

The history edits that trip the check are the same ones that restart the [prompt cache](#): injecting and removing per-turn reminders, summarizing older turns in place, or changing the system prompt mid-session. Send per-turn reminders as [turn-scoped system messages](#), change instructions or tools with a [mid-conversation system message](#) instead of rewriting `system` or `tools`, and let server-side [compaction](#) or [context editing](#) do any trimming. If you compact on the client, the simplest shape is to replace the whole history with one summary message plus the new user turn and replay nothing else: no thinking blocks carry over, so nothing fails, and the model thinks afresh on the compacted conversation (see [Custom compaction on the client](#)). Because cache reads are now cheaper (see [Pricing](#)), compacting early to save cost may no longer be the right cost-intelligence tradeoff on Claude Fable 5.1, so experiment with later compaction points.

To find edits your harness already makes, run a session with `prefix_mismatch_behavior: "drop_block"` and log `input_transformations`, as described in [How to tell whether your integration is impacted](#), or capture the exact requests it sends over a few normal turns and confirm that consecutive requests are byte-identical up to the appended turns.

Writing density

Claude Fable 5.1's writing has few stock phrases and little unexplained jargon. In some cases, though, its prose is denser than Claude Fable 5's: sentences run longer and there are fewer paragraph breaks. An instruction that defines the anti-pattern, mannered prose, helps. Add it to a user message (preferred) or the system prompt:

```
Mannered prose substitutes metaphor and flourish for direct statement. Instead of "a parameter worth varying," the mannered writer produces "a dial worth turning." Instead of "this point still matters," they write "this point earns its keep." The phrases exist to display the writer, not to convey the idea, and readers can tell. That is why mannered prose irritates: it makes the reader work harder so the writer can perform. It is also imprecise. Metaphors drag in connotations the writer did not choose and cannot control. The fix is to say what you mean. When a literal phrase is available, use it.
```

The short version also tends to work:

```
Please remove all mannered prose.
```

Formatting in chat

Earlier models overused bullets and bold in chat, and many prompts carry anti-formatting rules written to hold that down. Claude Fable 5.1 leans the other way: it uses bold less and is less likely to reach for headers, lists, or quotation marks. If your prompt contains anti-formatting language, remove it or replace it with a rule that says when specific formatting is appropriate, such as the following:

```
Use lists and bullet points when asked to, or when the content is multifaceted enough that they help with clarity. If the person explicitly requests minimal formatting, always format your responses without bullet points, headers, lists, or bold emphasis, as requested. In conversational, personal, or emotional exchanges, keep to plain prose.
```

Quoting retrieved sources

When summarizing documents, Claude Fable 5.1 is more likely than Claude Fable 5 to reproduce passages of the source text without marking them as quotations. To address this,

add one complete example of a correct response to the system prompt: the user's request, the response, and a sentence explaining why the response is correct.

```
<example>
<user>look up how the Riverton Ledger and the Coast Dispatch each covered the
Harbor Bridge closure and compare their reporting</user>
<response>
[web_search: Harbor Bridge closure Riverton Ledger]
[web_search: Harbor Bridge closure Coast Dispatch]
Both outlets agree on the basics: the bridge closed on March 3 after inspectors
found cracked welds, and the state expects repairs to take about eight months.
Where they differ is emphasis. The Ledger treats it as a local-economy story.
The Dispatch frames it as a funding failure; its editorial calls the closure
"entirely foreseeable." Read together, the Ledger explains who is affected now
and the Dispatch explains how it came to this – neither account alone gives the
whole picture.
</response>
<rationale>CORRECT: The response is organized around where the two outlets
agree and differ, not as a walk through either article. Each outlet's reporting
is conveyed in one or two sentences of the assistant's own indirect speech. One
short marked phrase from one source; every other claim is reworded. The
response is still specific and complete.</rationale>
</example>
```

Replace the two `[web_search: ...]` lines with your own tool's name, so the model reads them as templated tool output rather than literal text to emit.

Finish the whole task

Claude Fable 5.1 can execute very long tasks without much guidance on methodology, especially when the goal is clear. On complex asynchronous workloads, though, nudge it not to end its turn before the work is done. Without the nudge, the model sometimes describes what it would do next instead of doing it ("Next, I'll ...") or stops to ask permission for a step the original request already covered ("Shall I apply this?"). Users have to reply "continue" or "go ahead," which suits pair programming and other human-in-the-loop work but doesn't use the model's full long-horizon capability.

Two system prompt additions together mitigate this. Apply both. If you need to limit prompt length, use only the first, which keeps most of the effect. The first tells the model not to ask about work already requested and to carry out the next steps it has stated:

```
You are operating autonomously. The user is not watching in real time and
cannot answer questions mid-task, so asking 'Want me to...?' or 'Shall I...?' will
block the work. For reversible actions that follow from the original request,
proceed without asking. Stop only for destructive actions or genuine scope
changes the user must decide. Offering follow-ups after the task is done is
fine; asking permission before doing the work is not.
```

Exception: when the user is describing a problem, asking a question, or thinking out loud rather than requesting a change, the deliverable is your assessment. Report your findings and stop. Don't apply a fix until they ask for one.

Before ending your turn, check your last paragraph. If it is a plan, an analysis, a question, a list of next steps, or a promise about work you have not done ('I'll...', 'let me know when...'), do that work now with tool calls. That includes retrying after errors and gathering missing information yourself. Do not stop because the context or session is long. End your turn only when the task is complete or you are blocked on input only the user can provide.

Before running a command that changes system state (such as restarts, deletes, or config edits), check that the evidence actually supports that specific action. A signal that pattern-matches to a known failure may have a different cause.

The opening sentence, which tells the model the user isn't watching, carries much of the effect. Keep it as written. If your product needs the model to stop for specific confirmations, add a sentence after it listing them. This block can also make the model less likely to ask about ambiguous requests, so check that trade-off on your own tasks.

The second defines the user's request as the scope of the deliverable:

Delivering work

The user's request – or the plan they approved – sets the scope, and the scope is the deliverable: don't quietly narrow, widen, or swap it. Read ambiguity the way a careful colleague would: make routine judgment calls yourself, and check in only when different readings would lead to materially different work. If you see a real problem with the task as specified, say so in a sentence or two and keep building under stated assumptions; if the user hears the concern and reaffirms, that is their decision, so deliver the full request.

If a question comes up partway, first do everything that doesn't depend on the answer; then state the assumption you made, or – when going ahead on a wrong guess would be unsafe or would make the work useless – put the question at the end of a turn that also delivers that progress. If one part turns out to be blocked, complete every other part in full and say exactly what you left out and why – the whole task is the deliverable, and scaling it down is the user's call, not yours. A step you have decided on is something to run, not to announce: describing the next step and ending the turn leaves it undone until the user replies.

Keep changes to what the request needs. Something else you notice worth doing – cleanup or documentation the task didn't call for, a change to a file the task didn't require – is a suggestion to make at the end, not a change to make; actions clearly beyond what the ask implies, and risky or destructive ones, still need the user's go-ahead.

Tell the model what to preserve in compaction summaries

Claude Fable 5.1 responds well to being told explicitly what its summary must retain when a long conversation is compacted. Server-side [compaction](#) already does this. If you compact on the client side, use the following summarization instruction:

```
Summarize the transcript inside <summary></summary> tags. Include relevant information in the summary such that this conversation will be continued by a new context window without needing to redo work or be reprovided with relevant constraints or context. Be sure to preserve: (1) any difficulties or problems that came up, and how they were handled or resolved; (2) any possibilities, options, or approaches that were raised, tried, or set aside, and why; (3) anything that was asked for, decided, agreed, ruled out, or established as a preference, constraint, or boundary – stated exactly; (4) exactly where things stand now – what has been covered, settled, or completed so far; (5) anything still open, unresolved, promised, or expected to happen next; (6) specific details that would be hard to reconstruct – names, numbers, dates, exact wording, links or references – kept exactly. Be complete on these even at the cost of length; keep everything else concise. Weight the two voices differently: keep what the user said, asked for, shared, or established carefully and close to their own words; your own explanations and reasoning can be condensed much further, to what they concluded or produced – as long as nothing in the six items above is dropped.
```

Keep changes and tests to what the task asks for

When asked to implement an open-ended feature, Claude Fable 5.1 delivers what's asked for and sometimes more: it may fix nearby code, extend behavior the task didn't mention, or commit more test files than the change warrants. It responds well to explicit instructions about what to leave out. With the following instruction, unrequested additions and committed test code drop substantially with no measurable change in task success:

```
If, while working or testing, you find a pre-existing bug, a performance concern, or behavior the task doesn't mention, don't fix, optimize or extend it in this change unless the requested behavior cannot work without it; report it as a follow-up in your summary. Where the task is ambiguous, implement the reading its wording and the surrounding code most directly support, state that assumption in your summary, and don't build for the other readings as well. Verify your work however you like; scratch scripts and quick checks need not be kept. Commit tests only where the task asks for them or this repository already keeps tests for this kind of change, sized like the neighboring test files – roughly one focused test per stated behavior – and don't turn scratch checks into additional permanent test files. This is about extras only: implement every behavior the task asks for, completely.
```

Search triggering at low effort

At `low` effort, Claude Fable 5.1 is less likely than Claude Fable 5 to call a search or retrieval tool, and more likely to answer from memory. In some cases the simplest fix is to raise effort for the affected turns rather than the whole conversation. See [Change effort mid-conversation](#).

In other cases, a prompt nudge toward verification helps. In the system prompt, say that recognizing a name isn't the same as knowing its current state, and that such names should be searched as the user wrote them:

When a query centers on a name you do not confidently recognize, or recognize from a fast-moving area like AI models and developer tools where the landscape shifts within months, the name itself is the thing to verify: search before answering, and include the name as the user wrote it in at least one query alongside any reformulations. This holds even when you have some background on it – partial background is exactly what makes an out-of-date answer sound authoritative, so familiarity is not a reason to skip the search.

Reduce safeguard false positives

Claude Fable 5.1's safety classifiers produce fewer false positives than Claude Fable 5's did at launch, and finding vulnerabilities in source code is permitted. False positives still occur, and a blocked request returns `stop_reason: "refusal"` (see [Refusals, fallback, and billing](#)). Three situations make them more likely:

- **Compile-check phrasing:** Instead of "Does this program compile without errors?", ask "Are there any bugs in this program?"
- **Lesser-known programming languages:** Give the model context about what the language is and how it works, for example by giving it access to the language's documentation.
- **Base64 in tool output:** Tools that return base64-encoded data into the model's context can trigger false positives, so removing them is the recommended fix.

Prefer targeted edits over whole-file rewrites

If Claude Fable 5.1 rewrites whole files for small changes, append the following instruction to the system prompt or the first user message. Claude Fable 5.1 is more likely than Claude Fable 5 to rewrite an entire text file rather than make a targeted edit. The resulting file is usually the same, but unless the file is short or most of it is changing, a rewrite costs more output tokens and time. The instruction brings Claude Fable 5.1 back in line with Claude Fable 5 for small and medium changes.

The number of tokens used to edit files is best minimized, all else being equal. Therefore, when it will not affect the end result, try to surgically

edit a file rather than rewrite the entire thing.

Leave room for long outputs at xhigh and max effort

At `xhigh` and especially `max` effort, Claude Fable 5.1 can think for longer before it starts writing its reply. When a single request asks for a long deliverable, such as a full rewrite of a long document, it may draft much of that deliverable in its thinking and then write it out again as the reply, which means a longer wait and more output tokens. The simplest approach is to run requests like these at `high`, the recommended starting point, and move to `xhigh` or `max` only where you've measured a quality gain (see [Consider all effort levels](#)). If you do run them at `xhigh` or `max`:

- Set `max_tokens` to leave room for the thinking and the reply, not just the reply length you expect.
- Append the following note to the end of the user message. It makes the thinking much shorter on prose and code requests. Replace `[max_tokens]` with the request's actual `max_tokens` value, for example 64,000.

Everything produced in one reply, including any reasoning or drafting done before the reply, counts toward a single limit of about `[max_tokens]` tokens. If that limit is reached before the reply is finished, the person receives a cut-off response and has to start over. Composing an entire output or deliverable in full as reasoning and then again as a reply would double the length of the turn without improving the result, so don't do that.

Instead, when the person has asked for a long or effort-intensive deliverable such as a multi-section document, a large table or dataset, or a complete code file, spend extra effort on understanding the request, checking the inputs the answer depends on, settling the structure and other difficult decisions, and otherwise using the reasoning space to reason and the output space to write an output. Usually it is not needed to draft an output multiple times.

Let the lead agent keep working while subagents run

If your coding agent lets Claude Fable 5.1 delegate work to subagents, don't force the lead agent to stop and wait for each one. On coding tasks, letting the lead continue while subagents run lowers average time to completion at similar quality, token usage, and cost. To set this up:

- Have the tool that starts a subagent return immediately.
- Pass each subagent's result back to the lead in a later `user` message once it's ready.
- Give the lead a separate tool it can call when it wants to wait for a result.

The model still often chooses to wait. The time savings come from the runs where it carries on with other work.

Give vision work tools to crop and zoom

Claude Fable 5.1 has better vision capabilities out of the box, and on complex visual inputs such as dense charts it does its best work when it can iteratively analyze, crop, and visually verify what it sees. To get the full benefit, run the model as an agent with access to a container that holds the raw images or videos and has basic image-processing libraries (such as PIL and OpenCV) pre-installed. If running a container is too much overhead, an image-cropping tool alone delivers most of the uplift: a tool that returns a chosen region of the image, cropped and enlarged, lets the model examine specific details in more depth and scales test-time compute with image tokens. The [crop tool recipe](#) has a working definition.